Cours 2 Descente du Gradient

Ferdinand KAHENGA

École Supérieure d'Informatique SALAMA

Mars 2021

Plan du chapitre

- Introduction
- Étapes à suivre
- Fonction coût
- Calcul du gradient
- Minimisation de la fonction coût
- Exemple d'exécution de l'algorithme
- Implémentation de l'algorithme
- Généralisation et Vectorisation

Introduction

Dans ce cours nous allons parler de l'un des algorithmes d'optimisation au centre du Machine Learning. L'algorithme de descente du gradient(Gradient Descente). Il permet de trouver le minimum d'une fonction convexe (fonction avec un minimum global).

Introduction

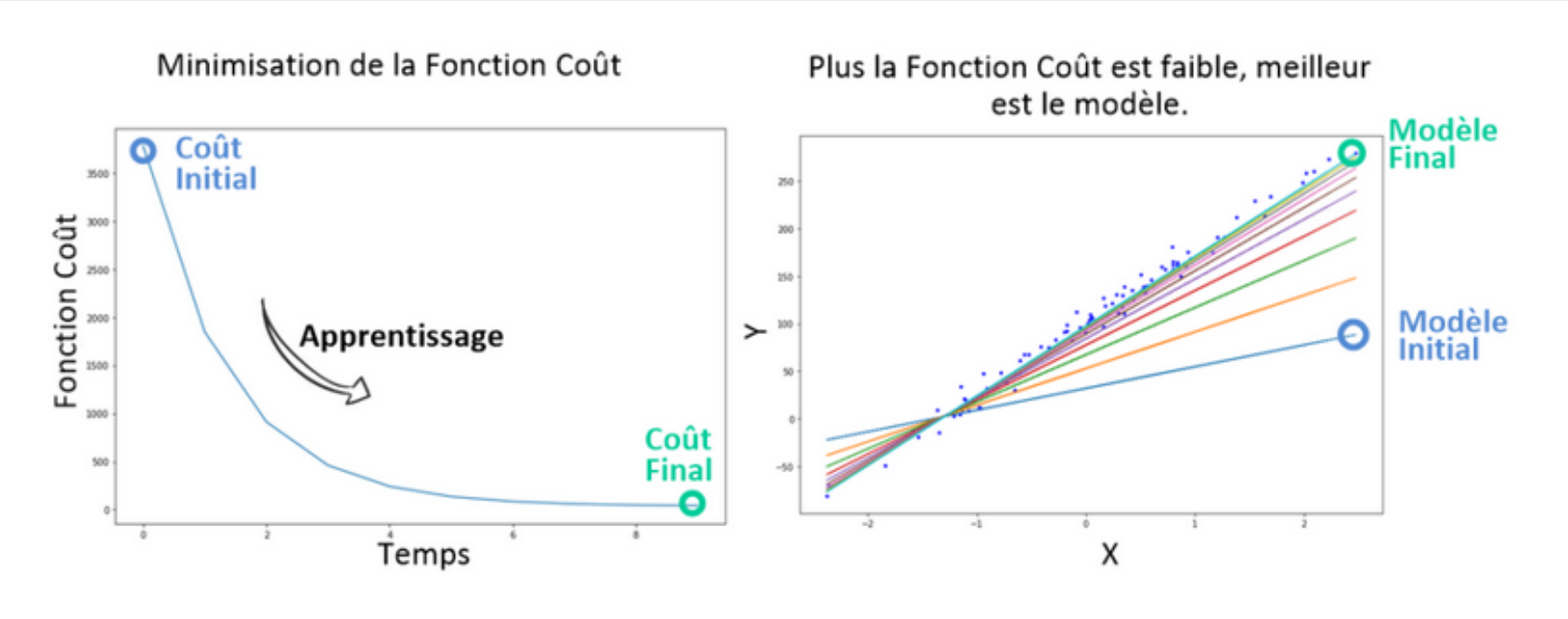

Figure – source :machinelearnia

Étapes

1. Trouver la fonction coût
2. Calculer le gradient de la fonction coût
3. Déterminer un learning rate α
4. Itérer pour minimiser la fonction coût

Problème

Supposons que nous avons $X = [0, 1, 4, 9, 6]$ et $Y = [1, 3, 9, 19, 13]$ et on demande de deviner la règle qui existe entre X et Y en utilisant la régression linéaire.

Notation

- Valeur réelle : y_i. Exemple pour $x_i = 4 \rightarrow y_i = 9$
- Prédiction : $\hat{y}_i = f(x_i) = a * x_i + b$
- Erreur : $L(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2 = (a * x_i + b - y_i)^2$
- Coût : $J(a, b) = \frac{1}{2*n} * \sum_{i=1}^{n} (\hat{y}_i - y_i)^2$

Calcul du gradient

La fonction coût est : $J(a,b) = \frac{1}{2*n} * \sum_{i=1}^{n} (\hat{y}_i - y_i)^2$.
J est fonction de a et b.
Le calcul du gradient va consister à faire des dérivées partielles en fonction de a et de b.

- $\frac{\partial J(a,b)}{\partial a} = \frac{1}{2*n} * \sum_{i=1}^{n} \frac{\partial (\hat{y}_i - y_i)^2}{\partial a}$
- $\frac{\partial J(a,b)}{\partial b} = \frac{1}{2*n} * \sum_{i=1}^{n} \frac{\partial (\hat{y}_i - y_i)^2}{\partial b}$

Calcul du gradient (suite)

Après développement on a :

- $\frac{\partial J(a,b)}{\partial a} = \frac{1}{n} * \sum_{i=1}^{n} x_i * (\hat{y}_i - y_i)$
- $\frac{\partial J(a,b)}{\partial b} = \frac{1}{n} * \sum_{i=1}^{n} (\hat{y}_i - y_i)$

Algorithme

Au départ donner des valeurs aléatoires à a et b, puis faire les itérations suivantes

$$\begin{cases} a = a - \alpha \frac{\partial J(a,b)}{\partial a} \\ b = b - \alpha \frac{\partial J(a,b)}{\partial b} \end{cases}$$

Un bon α est autour de 0.001

Test de l'algorithme

X	0	1	4	9	6
Y	1	3	9	19	13

Table – Données d'exemple

Initialiser :

- $a = 4$ et $b = 3$
- $\alpha = 0.001$
- $epoch = 10$

```
def dg_optimisation(A,B,alpha,nbEpoch,m):
    #A et B sont des parametres
    #nbEpoch: epochs, m:nbre occurrences
    for epoch in range(nbEpoch):
        Yhat,J=0,0
        dA,dB=0,0
        for i in range(m):
            Yhat=A*X[i]+B
            J+=(Yhat-Y[i])**2
            dA+=X[i]*(Yhat-Y[i])
            dB+=(Yhat-Y[i])
        dA=dA/m
        dB=dB/m
        J=J/(2*m)
        A=A-alpha*dA
        B=B-alpha*dB
        print("epoch",epoch+1,"=========>J:",J, "A=",A,"B=",B)
```

Généralisation

- Jusque là nous avons fait le développement sur un dataset comportant :
  - 1 facteur
  - n occurrences
- Comment généraliser les équations sur un dataset comportant :
  - m facteurs
  - n occurrences

Généralisation

Target y	Features x_1	x_2	x_3
Prix	Surface m2	N chambres	Qualité
€ 313,000.00	124	3	1.5
€ 2,384,000.00	339	5	2.5
€ 342,000.00	179	3	2
€ 420,000.00	186	3	2.25
€ 550,000.00	180	4	2.5
€ 490,000.00	82	2	1
€ 335,000.00	125	2	2

- Notons $X^{[i]}$: l'un des facteurs. Avec $i \in [1, m]$.
- Notons $X_j^{[i]}$: le j^e élément du facteur $X^{[i]}$. Avec $j \in [1, n]$
- Exemple : $X_2^{[1]} \rightarrow 339$, $X_4^{[2]} \rightarrow 3$, $X_7^{[3]} \rightarrow 2$

Modèle

- On aura m paramètres A_i à calculer et le biais B
- Prédiction : $\hat{Y}_j = A_i * X_j^{[i]} + B$ Avec i allant de 1 à m.
  Autrement
  $\hat{Y}_j = A_1 * X_j^{[1]} + A_2 * X_j^{[2]} + ... + A_m * X_j^{[m]} + B$
- Erreur : $L(\hat{Y}_j, Y_j) = (\hat{Y}_j - Y_j)^2$
- Coût : $J(A, B) = \frac{1}{2*n} * \sum_{j=1}^{n} (\hat{Y}_j - Y_i)^2$

Calcul du gradient

Après développement on a :

- $\frac{\partial J(A,B)}{\partial A_i} = \frac{1}{n} * \sum_{j=1}^{n} X_j^{[i]} * (\hat{Y}_j - Y_j)$
- $\frac{\partial J(A,B)}{\partial B} = \frac{1}{n} * \sum_{j=1}^{n} (\hat{Y}_i - Y_i)$

Algorithme

Au départ donner des valeurs aléatoires à A_i et B, puis faire les itérations suivantes

$$\begin{cases} A_i = A_i - \alpha \frac{\partial J(A,B)}{\partial A_i} \\ B = B - \alpha \frac{\partial J(A,B)}{\partial B} \end{cases}$$

Un bon α est autour de 0.001

Algorithme

Proposer un code python permettant de généraliser la fonction *gd_optimisation*.

Vectorisation

- La vectorisation permet d'exprimer l'équation du modèle sous forme matricielle
- Son avantage est d'utiliser les libraires optimisées comme *Numpy* pour exécuter rapidement
- Eviter si possible d'utiliser la boucle

Modèle

Mettons les paramètres et les facteurs dans deux matrices appelées respectivement A et X.

$$A = \begin{pmatrix} A_1 \\ A_2 \\ .. \\ A_m \end{pmatrix}, X = \begin{pmatrix} X_1^{[1]} & X_2^{[1]} & X_3^{[1]} & .. & X_n^{[1]} \\ X_1^{[2]} & X_2^{[2]} & X_3^{[2]} & .. & X_n^{[2]} \\ .. & .. & .. & .. & .. \\ X_1^{[m]} & X_2^{[m]} & X_3^{[m]} & .. & X_n^{[m]} \end{pmatrix}$$

- A est une matrice colonne de taille $(1, m)$ et X est une matrice de taille (m, n)
- Le modèle devient : $\hat{Y} = A^T X + B$

Code Python

```
X=np.array([[0, 1,  4, 9, 6]])
Y=2*X+1

A=np.array([[4]])
B=3

nbEpoch=10
alpha=0.01
m=X.shape[1]

for i in range(nbEpoch):
    Yhat=np.dot(A.T,X)+B
    dZ=Yhat-Y
    cost = 1/(m*2)*np.sum((Yhat-Y)**2)
    print("epoch=>",i+1," A=>",A[0][0]," B=>",B," cost=>",cost)
    dA=(1/m)*np.dot(X,dZ.T)
    dB=(1/m)*np.sum(dZ)
    A=A-alpha*dA
    B=B-alpha*dB
```